

# Inteligência Artificial

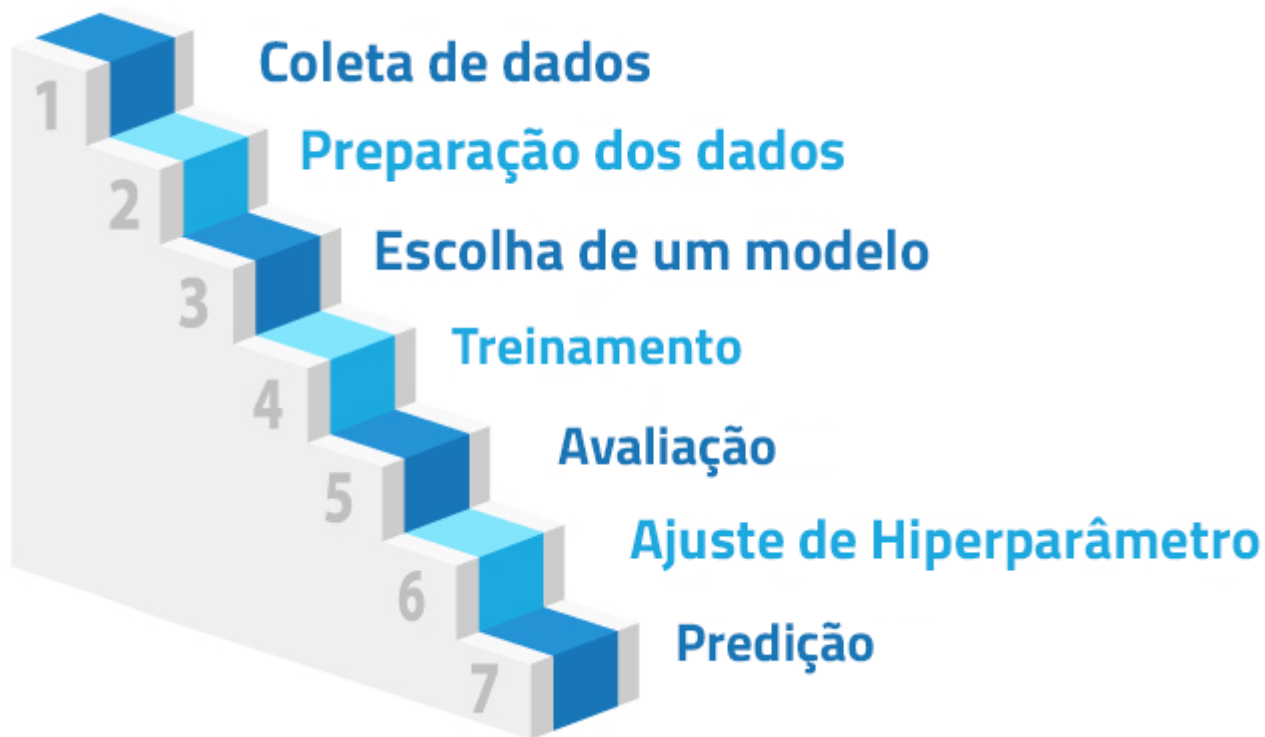
Prof<sup>a</sup>. Victoria Dala Pegorara Souto



# MACHINE LEARNING

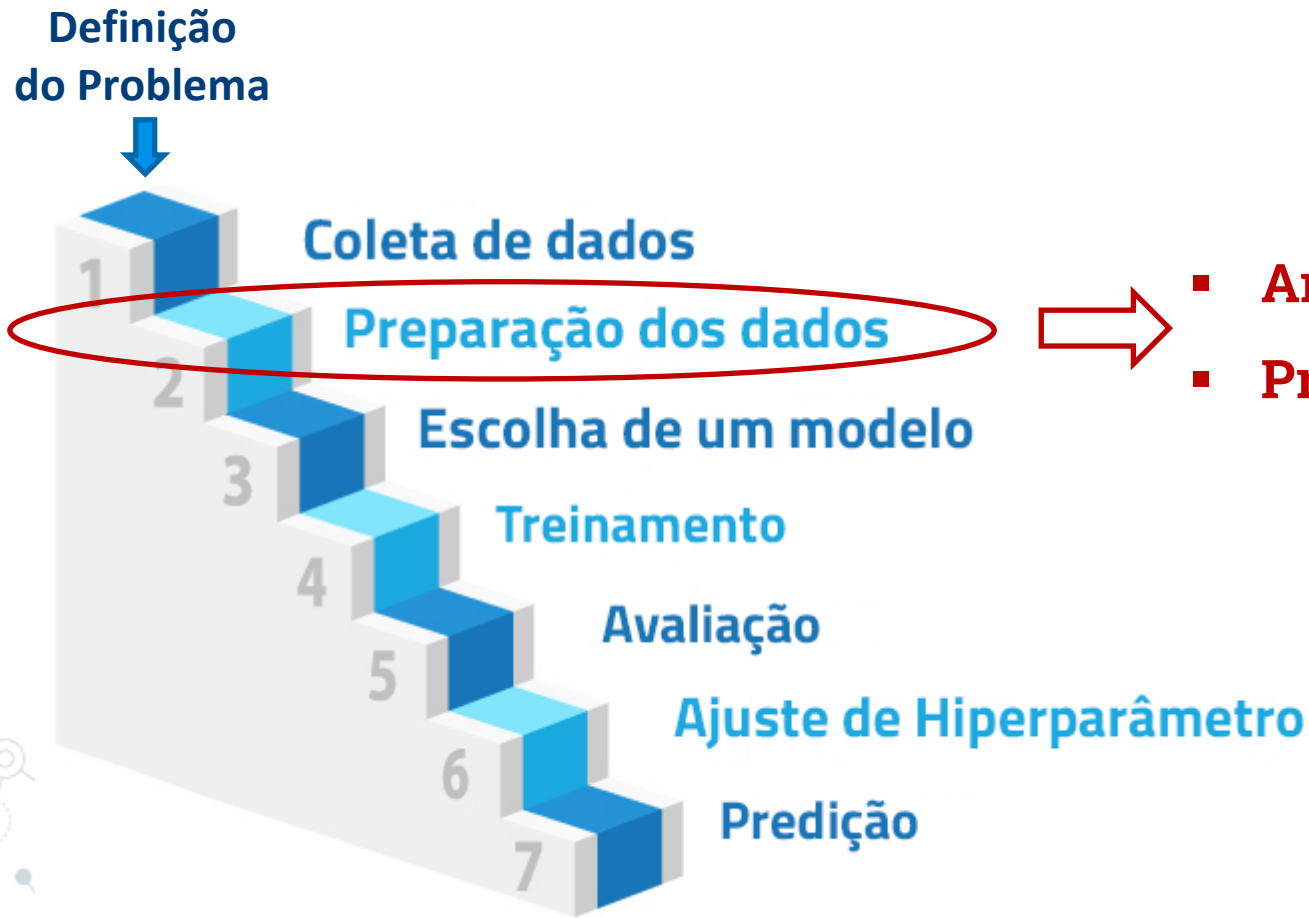
## Etapas do Projeto de ML

Definição  
do Problema



# MACHINE LEARNING

## Etapas do Projeto de ML



- **Análise Exploratória de Dados (EDA)**
- **Pré-processamento dos Dados.**

## DADOS BAGUNÇADOS: O caos que você conhece



## DADOS PRONTOS PARA O MODELO: Música para os ouvidos da IA!



A jornada de todo cientista de dados: da bagunça à beleza

**Continuamos na jornada de preparação dos dados!**



# PRÉ-PROCESSAMENTO DE DADOS

## → TÉCNICAS:

- Divisão do *Dataset*,
- Escalonamento de atributos,
- Codificação de atributos,
- Engenharia de atributos,
- Extração de atributos,
- Seleção de atributos.



# PRÉ-PROCESSAMENTO DE DADOS

## → **DIVISÃO DO *DATASET*:**

Em ML, o objetivo não é apenas aprender os dados disponíveis, mas generalizar bem para dados novos (i.e., inéditos).

→ **Para isso, o *dataset* é dividido em três subconjuntos:**

- ✓ Treinamento
- ✓ Validação
- ✓ Teste

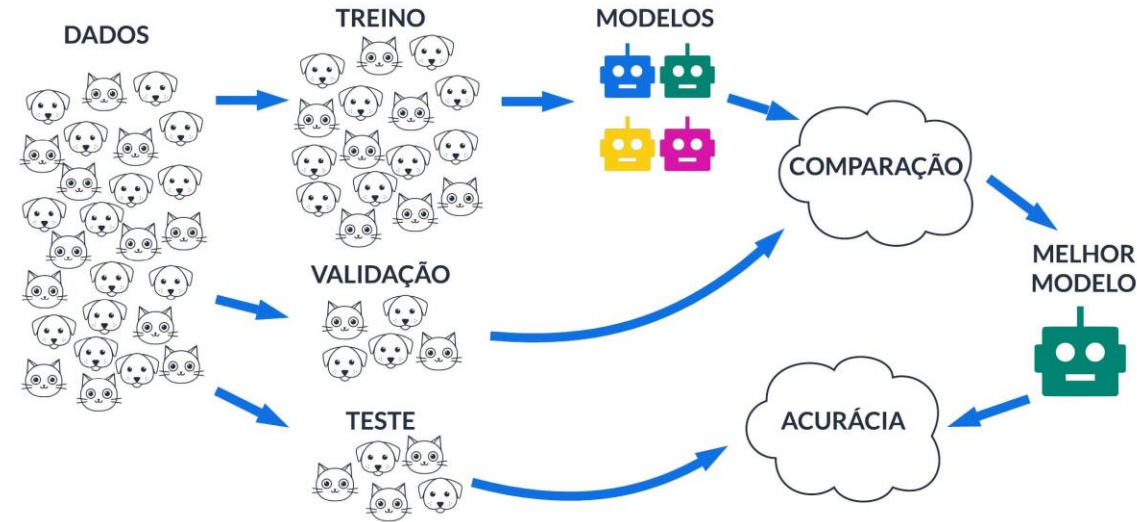
Não é correto usar o mesmo conjunto de dados para treinamento, validação e teste!

# PRÉ-PROCESSAMENTO DE DADOS

## → DIVISÃO DO *DATASET*:

**Conjunto de Treinamento:** Usado para **treinar** os modelos de ML.

- É onde o modelo “aprende” os padrões dos dados.
- Quanto mais representativo, melhor o aprendizado.
- Geralmente é a maior parte dos dados (por exemplo, 60–90%).



**Conjunto de Validação:** Usado para **escolher o melhor modelo e ajustar hiperparâmetros** (ex.: profundidade da árvore, taxa de aprendizado, regularização, número de neurônios).

**Como Utilizar?** Você treina no **treinamento** e compara desempenho na **validação** para decidir configurações sem “viciar” no teste. **Evita que o modelo decore os dados de treinamento (i.e., se sobreajuste/overfit).**

- Ajustar hiperparâmetros.
- Escolher arquiteturas.
- Decidir quando parar o treinamento (*early stopping*).



# PRÉ-PROCESSAMENTO DE DADOS

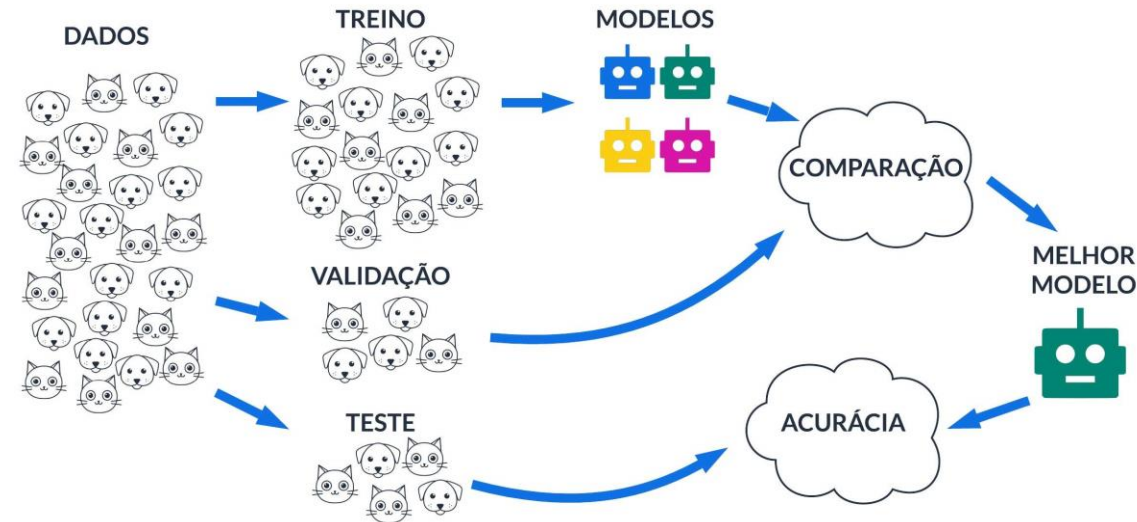
## → DIVISÃO DO *DATASET*:

**Conjunto de Teste:** Usado para **validar/testar** a capacidade de generalização dos modelos.

→ Usado uma única vez, ao final.

→ Fornece uma estimativa imparcial do desempenho **real** do modelo.

→ Simula dados nunca vistos.



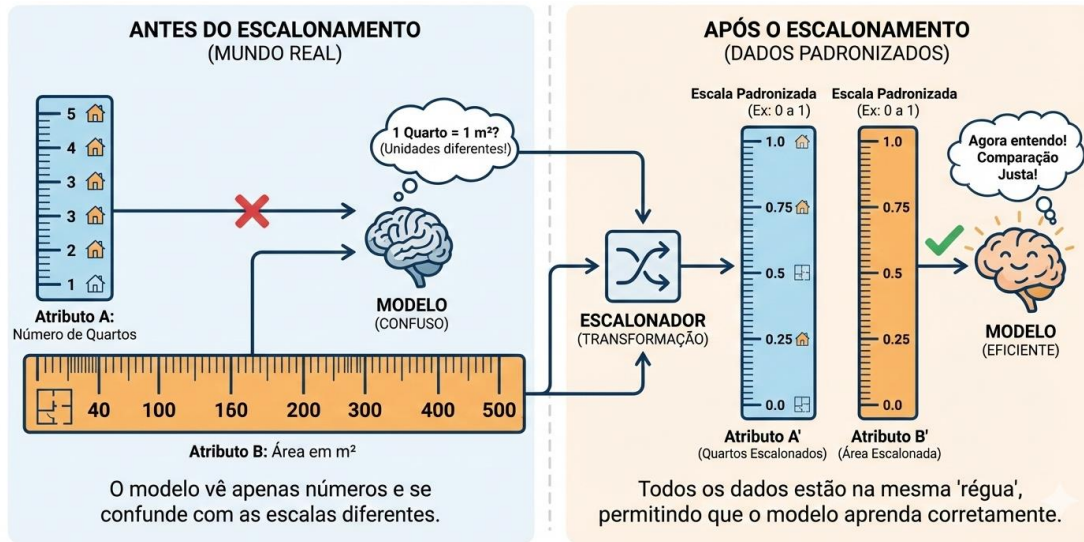
- Todos os três devem ser representativos do problema/fenômeno sendo analisado.
- Em geral, mas **não é regra**, divide-se o *dataset* original da seguinte forma:

| Treinamento | Validação | Teste |
|-------------|-----------|-------|
| 70%         | 15%       | 15%   |
| 80%         | 10%       | 10%   |



# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS



Precisamos colocar todos os dados na mesma régua (i.e., **escala**).

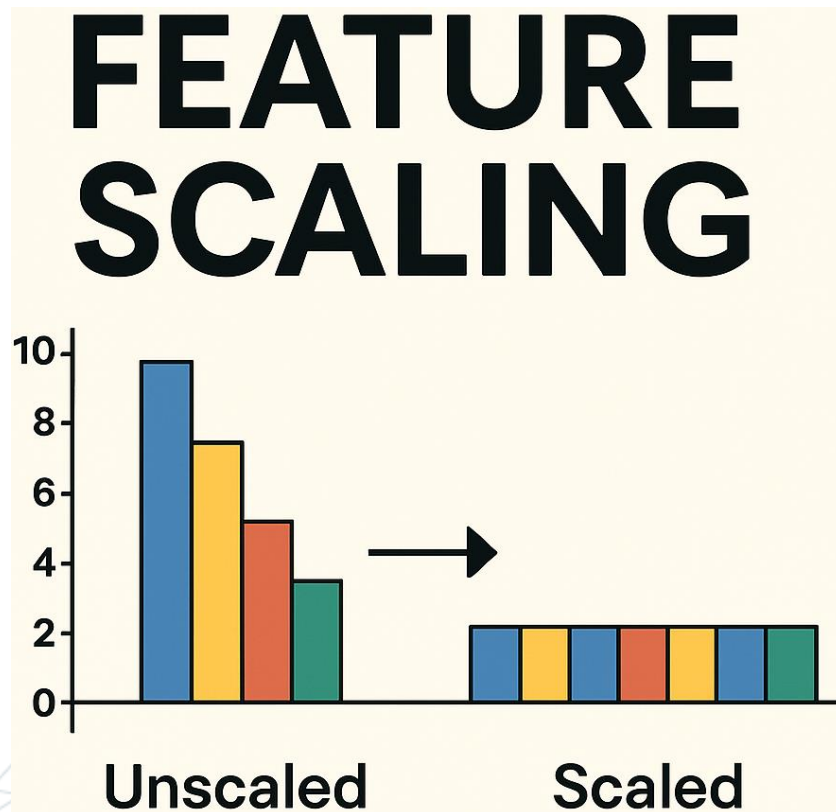
Imaginem criar um modelo para a previsão do preço de casas:

- Atributo A: Número de quartos (1 a 5).
- Atributo B: Área em  $m^2$  (40 a 500).
- Para o modelo, 1 quarto é igual a 1  $m^2$ .

Os modelos não conhecem as unidades do mundo real, eles conhecem apenas números!

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS



### Por Que Escalonar Atributos?

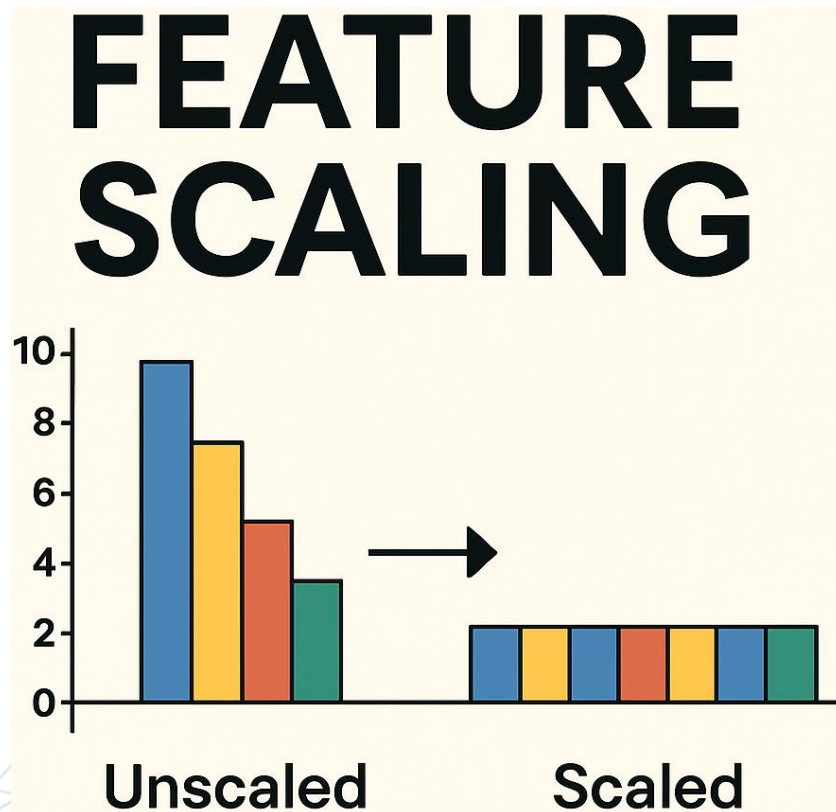
Modelos de ML baseados em **distância** (e.g., KNN, SVM, PCA) ou **gradiente** (e.g., regressão linear/logística, redes neurais) são sensíveis à escala dos atributos.

Atributos com escala muito maior influenciam muito mais a previsão dos modelos do que atributos com escala menor.

- ✓ No exemplo da previsão do preço de casas, o modelo ignoraria os quartos e focaria só na área.

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS



### Por Que Escalonar Atributos?

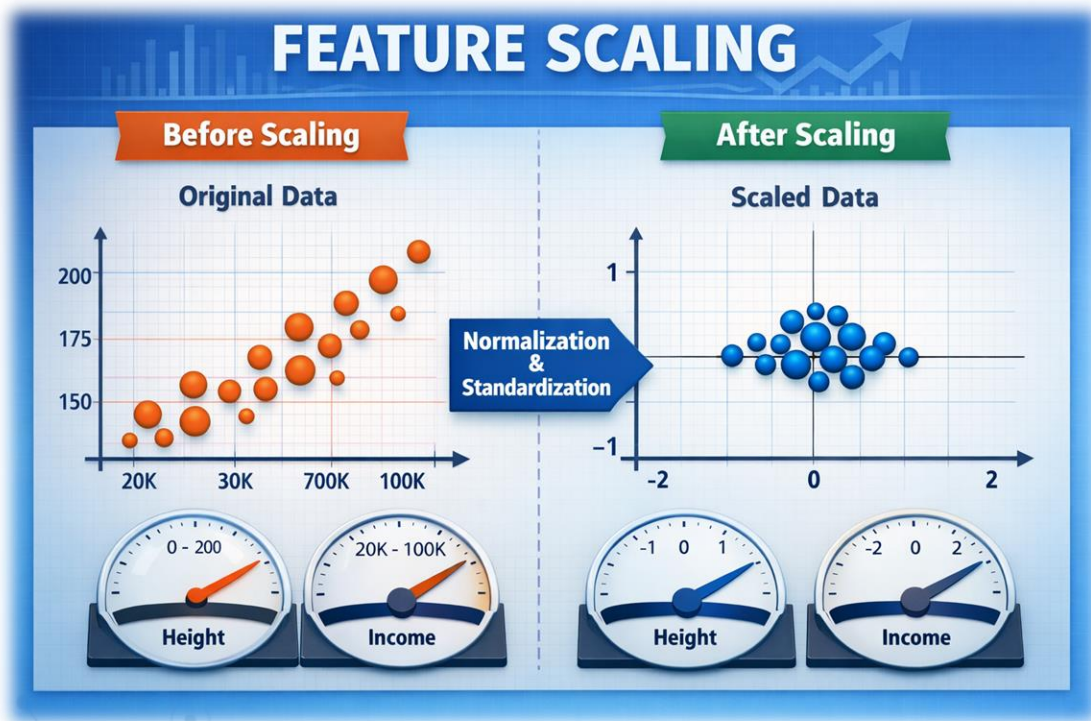
Mas e se todos os atributos são importantes para a previsão do modelo?

→ O escalonamento iguala a influência dos atributos no modelo.

Escalonamento transforma valores de atributos para uma mesma escala, geralmente sem unidade definida.

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS



### Principais Técnicas de Escalonamento

- ✓ Padronização
- ✓ Normalização
- ✓ Escalonamento robusto



# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS

**Normalização (Min-Max Scaling)** → Transforma todos os valores numéricos para uma escala fixa, geralmente entre 0 e 1.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1],$$

onde  $x$  é o valor original,  $x_{\min}$  e  $x_{\max}$  são os valores mínimo e máximo de uma coluna (i.e., atributo).

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler(feature_range=(0, 1))
```

 ⇒ Instância do normalizador com intervalo de 0 a 1

```
X_scaled = scaler.fit_transform(X_train)
```

 ⇒ Aprende os parâmetros de normalização no treino

```
X_test_scaled = scaler.transform(X_test)
```

 ⇒ Aplica o scaler aprendido no teste

# PRÉ-PROCESSAMENTO DE DADOS

## → Normalização (Min-Max Scaling)

### → Quando usar??

- ✓ Dados sem *outliers* (analisar e remover antes),
- ✓ Com algoritmos que não pressupõem nenhuma distribuição (Ex.: redes neurais, KNN) e/ou que exigem entradas limitadas como os baseados em distância (Ex.: KNN, SVM, K-Means) e gradiente (Redes Neurais, Regressões).

**Observação:** Muito sensível a *outliers*. Um *outlier* extremo “esmaga” todos os outros valores para um intervalo muito pequeno próximo de zero.

- ✓ Os valores (que não são *outliers*) acabam sendo transformados em valores muito pequenos, próximos de zero, perdendo a distinção entre eles.
- ✓ Isso reduz a eficácia do aprendizado, pois a diferença entre os valores reais é praticamente eliminada na escala normalizada.

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS

**Padronização (Standard Scaling)** → Transforma os atributos para terem média 0 e desvio padrão 1, não impondo limites.

$$x' = \frac{x - \mu_x}{\sigma_x},$$

onde  $x$  é o valor original,  $\mu_x$  e  $\sigma_x$  são a média e o desvio padrão de uma coluna (i.e., atributo).

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```



Instância do padronizador

```
X_scaled = scaler.fit_transform(X_train)
```



Aprende os parâmetros de padronização no treino

```
X_test_scaled = scaler.transform(X_test)
```



Aplica o scaler aprendido no teste

# PRÉ-PROCESSAMENTO DE DADOS

## → Padronização (Standard Scaling)

### Vantagens:

- ✓ Torna os dados "comparáveis" em termos do número de desvios padrão da média.
- ✓ Mais robusto a *outliers*, mas não a *outliers* extremos.
- ✓ Por não dividir os valores por um denominador muito grande, no caso da presença de *outliers*, não os transforma em valores próximos de zero e, portanto, não se perde a distinção entre eles.

### Quando usar?

- ❖ Em geral, com algoritmos que assumem distribuição "normal" ou Gaussiana (e.g., GNB, GMM, alguns modelos para detecção de anomalias), e quando os dados possuem *outliers*.



# PRÉ-PROCESSAMENTO DE DADOS

## → Padronização (Standard Scaling)

### Observação:

- Se os dados originais são assimétricos, eles continuarão assimétricos, apenas centralizados e com variância igual a 1.
- Se o conjunto de dados tiver *outliers* extremos, o **Escalonador Robusto** é uma opção superior à padronização, pois **ignora os valores discrepantes no cálculo do escalonamento**.
- *Outliers* podem influenciar a média e variância dos atributos de forma negativa.

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS

**Escolonamento Robusto (Robust Scaling)** → Usa mediana e distância interquartil no escalonamento.

$$x' = \frac{x - \text{mediana}}{\underbrace{Q_3 - Q_1}_{\text{IQR}}},$$

onde  $x$  é o valor original e IQR é a distância interquartil de um atributo.

```
from sklearn.preprocessing import RobustScaler
```

```
scaler = RobustScaler()
```



Instância do escalonador robusto

```
X_scaled = scaler.fit_transform(X_train)
```



Aprende os parâmetros de padronização no treino

```
X_test_scaled = scaler.transform(X_test)
```



Aplica o scaler aprendido no teste

# PRÉ-PROCESSAMENTO DE DADOS

## → Escalonamento Robusto (Robust Scaling)

### Vantagens:

- Por utilizar a mediana e o IQR em vez da média e do desvio padrão, os *outliers* têm influência mínima nos dados escalonados.
- Mantém a relação de distância entre os valores não-discrepantes melhor que a padronização quando existem *outliers* extremos.
- Ou seja, não deixa que *outliers* extremos esmaguem os valores não-discrepantes em valores próximos de 0.

### Quando usar?

- Quando há *outliers* significativos, mas desejamos mantê-los sem impactar o escalonamento.
- Quando as distribuições são assimétricas ou com caudas longas.

### Cuidado:

- Como não remove os *outliers*, algoritmos sensíveis à distância (e.g., KNN, Regressores) podem ter seu aprendizado impactado.
- A escala dos dados não discrepantes é mantida, mas ele não esconde os *outliers*.

# PRÉ-PROCESSAMENTO DE DADOS

## → ESCALONAMENTO DE ATRIBUTOS

### Armadilhas durante o Escalonamento de Atributos:

- Para evitar vazamento de dados, o **escalador deve sempre ser ajustado no conjunto de treinamento**.
- Os conjuntos de validação e teste, são transformados com os valores encontrados com o conjunto de treinamento.
- Geralmente, não se aplica escalonamento à **variável alvo**, i.e., os rótulos, apenas aos atributos.
- Não se deve escalonar atributos binários ou *one-hot* sem necessidade, pois pode-se criar valores fracionários desnecessários.

# PRÉ-PROCESSAMENTO DE DADOS

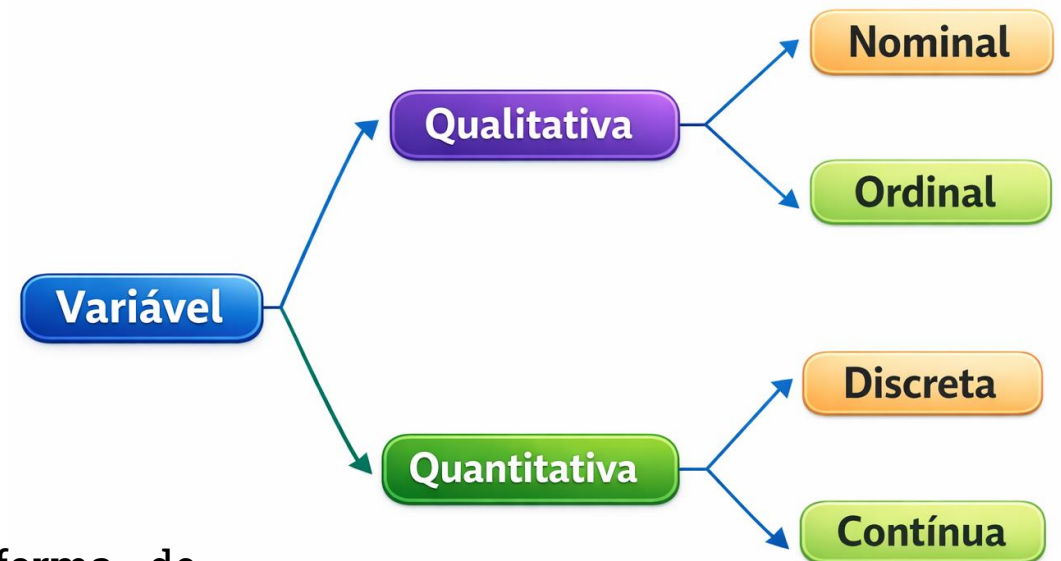
## → CODIFICAÇÃO DE ATRIBUTOS CATEGÓRICOS

→ Modelos matemáticos não entendem texto, portanto, precisamos converter categorias em números.

**As principais técnicas de codificação são:**

- ✓ One-Hot encoding
- ✓ Label encoding
- ✓ Target encoding
- ✓ Binary encoding

**Observação:** A codificação pode ser considerada uma forma de engenharia de atributos.





# PRÉ-PROCESSAMENTO DE DADOS

## → One-Hot Encoding

- ✓ Usada com atributos nominais (i.e., sem ordem) e com baixa cardinalidade (i.e., com poucas categorias distintas).
- ✓ Cria uma nova coluna binária para cada categoria distinta.

**Limitação:** Gera alta dimensionalidade (i.e., muitas colunas) se houver muitas categorias, aumentando o custo computacional.

- Pode criar o problema conhecido como a “maldição da dimensionalidade”.

One-Hot Encoding

| Color | Encoded |
|-------|---------|
| Red   | 1 0 0   |
| Blue  | 0 1 0   |
| Green | 0 0 1   |

[1 0 0] [0 1 0]

Label Encoding

| Color | Encoded |
|-------|---------|
| Red   | 1       |
| Blue  | 2       |
| Green | 3       |

1 2 3

Target Encoding

| Color | Encoded |      |
|-------|---------|------|
| Red   | 0.8     | 0.75 |
| Blue  | 0.4     | 0.5  |
| Green | 0.6     | 0.65 |

0.8 0.4 0.6

Binary Encoding

| Color | Encoded |
|-------|---------|
| Red   | 01      |
| Blue  | 10      |
| Green | 11      |

[01] [10] [11]

# PRÉ-PROCESSAMENTO DE DADOS

## → Label Encoding

- ✓ Converte cada categoria em um **número inteiro único**, mas sem considerar uma ordem.
- ✓ Indicada principalmente para a **codificação do rótulo** em problemas de classificação.

**Limitação:** Pode introduzir uma relação de ordem falsa que afeta negativamente algoritmos sensíveis a distâncias ou magnitudes (e.g., regressores linear e logístico e KNN).

- Para tamanho de camisetas (P, M, G), faz sentido (ordinal).
- Para animais e cores, não (nominal).

One-Hot Encoding

| Color | Encoded |
|-------|---------|
| Red   | 1 0 0   |
| Blue  | 0 1 0   |
| Green | 0 0 1   |

[1 0 0] [0 1 0]

Label Encoding

| Color | Encoded |
|-------|---------|
| Red   | 1       |
| Blue  | 2       |
| Green | 3       |

1 2 3

Target Encoding

| Color | Encoded  |
|-------|----------|
| Red   | 0.8 0.75 |
| Blue  | 0.4 0.5  |
| Green | 0.6 0.65 |

0.8 0.4 0.6

Binary Encoding

| Color | Encoded |
|-------|---------|
| Red   | 01      |
| Blue  | 10      |
| Green | 11      |

[01] [10] [11]

# PRÉ-PROCESSAMENTO DE DADOS

## → Target Encoding

Substitui cada categoria pela **média do valor do alvo** correspondente a essa categoria.

**Quando usar?** Ideal para variáveis de alta cardinalidade, pois não aumenta a dimensionalidade do conjunto de dados e quando há relação entre a categoria e o *target*.

**Risco:** *Overfitting* quando uma categoria tem poucos exemplos.

- A média do *target* pode refletir mais ruído do que padrão real dos dados, prejudicando a generalização.

| workclass        | target |   | workclass        | target mean |   | workclass |
|------------------|--------|---|------------------|-------------|---|-----------|
| State-gov        | 0      |   | State-gov        | 0           |   | 0         |
| Self-emp-not-inc | 1      |   | Self-emp-not-inc | 1           |   | 1         |
| Private          | 0      | → | Private          | 1/3         | → | 1/3       |
| Private          | 0      |   |                  |             |   | 1/3       |
| Private          | 1      |   |                  |             |   | 1/3       |

# PRÉ-PROCESSAMENTO DE DADOS

## → Binary Encoding

- ✓ Combina os benefícios das codificações *One-Hot* e *Label*.
- ✓ Cada categoria recebe um número inteiro (*Label*), e depois esse número é convertido para sua representação binária, criando colunas para cada bit.

### Quando usar?

→ Ótimo para variáveis categóricas de alta cardinalidade.

### Desvantagem:

→ Por algumas categorias compartilharem bits, modelos lineares podem achar que existe uma relação matemática entre elas. Modelos de árvore (e.g., DT, RF) lidam bem com isso.

| Color  |
|--------|
| Green  |
| Red    |
| Black  |
| Orange |

Binary  
encoding



| Color_0 | Color_1 |
|---------|---------|
| 0       | 0       |
| 0       | 1       |
| 0       | 1       |
| 1       | 0       |
| 1       | 1       |

# PRÉ-PROCESSAMENTO DE DADOS

## → CODIFICAÇÃO DE ATRIBUTOS CATEGÓRICOS

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, TargetEncoder

# SciKit-Learn não implementa o codificador binário (pip install category_encoders)
from category_encoders import BinaryEncoder

# LabelEncoder(), TargetEncoder() ou BinaryEncoder()
enc = OneHotEncoder()
enc.fit_transform(X_train)

enc.transform(X_test)
enc.inverse_transform(X_test)
```



# PRÉ-PROCESSAMENTO DE DADOS

## → ENGENHARIA DE ATRIBUTOS

→ **Definição:** É a transformação ou criação de novos atributos que sejam mais úteis e informativos.

→ **Por Que Fazer?** Ajuda o modelo a enxergar relações que não estão óbvias.

→ **Objetivo:** Melhorar a capacidade de predição (generalização) do modelo.

### Exemplos:

- Data de nascimento → Idade, faixa etária.
- Coordenadas GPS → Distância até certo ponto.
- Preço e quantidade → Valor total da venda.
- *Timestamp* → Hora, dia da semana, feriado?
- Comprimento e largura → Área.
- Interação entre atributos → Polinômios (e.g.,  $x'_1 = x_1x_2 + x_1^2x_2$ )

# PRÉ-PROCESSAMENTO DE DADOS

## → ENGENHARIA DE ATRIBUTOS

```
import pandas as pd
```

```
# Exemplo de dados
```

```
df = pd.DataFrame({  
    "preco": [10, 20, 15],  
    "quantidade": [2, 1, 3],  
    "desconto": [0, 5, 0]})
```



Cria um DataFrame chamado df a partir de um dicionário.

```
# Feature Engineering
```

```
df["valor_total"] = df["preco"] * df["quantidade"]
```



Cria uma nova coluna "valor\_total" multiplicando preço × quantidade em cada linha (multiplicação vetorizada do pandas)

```
df["houve_desconto"] = (df["desconto"] > 0).astype(int)
```



Cria uma coluna "houve\_desconto" que indica se existiu desconto.

# PRÉ-PROCESSAMENTO DE DADOS

## → EXTRAÇÃO DE ATRIBUTOS

→ Processo de **transformar dados brutos** em um conjunto menor de atributos **significativos** que podem ser processados de forma mais eficiente, mantendo as informações essenciais (em geral, perde parte da informação).

**OBJETIVO:** Criar representações mais úteis do que os dados originais, reduzindo ruído, destacando padrões e, muitas vezes, diminuindo a dimensionalidade.

### Vantagens:

- ✓ Reduz a dimensionalidade,
- ✓ Remove redundância,
- ✓ Reduz os recursos computacionais necessários para armazenamento, processamento e treinamento,
- ✓ Acelera o treinamento,
- ✓ Melhora o desempenho do modelo, pois os dados são mais informativos e menos ruidosos.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

→ **Dados tabulares:** Análise de componentes principais (PCA) e análise de componentes independentes (ICA).

- Criam novos atributos não correlacionados que capturam a variância máxima (PCA) ou a independência dos atributos (ICA).

**PCA:** Cria novos atributos (componentes) que são combinações lineares das variáveis originais, de modo que: o 1º componente captura a maior variância possível, o 2º captura a maior variância restante, sem se correlacionar com o 1º, e assim por diante.

- ✓ **Exemplo:** Imagine que seus dados estão “espalhados” no espaço. O PCA encontra os melhores eixos para “enxergar” esse espalhamento com o máximo de informação.
- ✓ **Objetivos:** Reduzir dimensionalidade (menos colunas, mais rapidez) e Remover redundância (variáveis muito correlacionadas)
- ✓ **Observações:** PCA não é supervisionado (não usa o rótulo), normalmente pede padronização antes (z-score), senão variáveis com escala maior dominam.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

→ **Dados tabulares:** Análise de componentes principais (PCA) e análise de componentes independentes (ICA).

- Criam novos atributos não correlacionados que capturam a variância máxima (PCA) ou a independência dos atributos (ICA).

**ICA:** Também cria novos atributos como combinações lineares, mas com outro objetivo → busca componentes estatisticamente independentes (mais forte que “não correlacionado”).

- ✓ **Exemplo:** é como tentar separar “fontes” misturadas, por exemplo, separar vozes de diferentes pessoas gravadas no mesmo microfone (*blind source separation*).
- ✓ **Objetivos:** Quando você suspeita que os dados são mistura de fontes independentes, sinais (bio-sinais, áudio), finanças, ou cenários onde “independência” faça sentido.

### Diferença principal:

**PCA:** maximiza variância e garante não correlação.

**ICA:** tenta recuperar fontes independentes (pode capturar estruturas não gaussianas melhor).



# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

→ **Texto:** *Bag of words* (BoW), *Term Frequency-Inverse Document Frequency* (TF-IDF), *Word Embeddings*.

- Transformam texto em representações numéricas (modelos de ML entendem números), variando de contagens simples de palavras, a pesos informativos, até vetores densos que capturam significado e contexto.

**BoW (Bag of Words):** Transforma texto em um vetor onde cada posição representa uma palavra do vocabulário e o valor é:

→ 0/1 (presença).

→ Contagem (quantas vezes aparece).

- ✓ **Intuição:** “ignora a ordem” e olha só para “quais palavras aparecem”.
- ✓ **Vantagens:** simples, rápido, funciona bem em tarefas como spam/assunto.
- ✓ **Limitações:** perde contexto e semântica; vocabulário pode ficar enorme.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

→ **Texto:** *Bag of words* (BoW), *Term Frequency-Inverse Document Frequency* (TF-IDF), *Word Embeddings*.

- Transformam texto em representações numéricas (modelos de ML entendem números), variando de contagens simples de palavras, a pesos informativos, até vetores densos que capturam significado e contexto.

**TF-IDF:** É como BoW, mas pondera palavras:

- **TF (Term Frequency):** aparece muito **no** documento → tende a ser **importante**.
  - **IDF (Inverse Document Frequency):** aparece em **muitos documentos** → tende a ser **pouco informativa** (ex.: “de”, “para”, “muito”).
- ✓ **Resultado:** Palavras **características** de um texto ganham mais peso do que palavras muito comuns.
- ✓ **Aplicação:** Classificação de textos, busca, temas, com modelos lineares (SVM, regressão logística).

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

→ **Texto:** *Bag of words* (BoW), *Term Frequency-Inverse Document Frequency* (TF-IDF), *Word Embeddings*.

- Transformam texto em representações numéricas (modelos de ML entendem números), variando de contagens simples de palavras, a pesos informativos, até vetores densos que capturam significado e contexto.

**Word Embeddings:** São uma forma de representar palavras (ou textos) como vetores numéricos densos (ex.: 50, 100, 300, 768 dimensões), onde a geometria do espaço captura significado: palavras com sentidos parecidos ficam próximas.

→ Em vez de “uma coluna por palavra” (como BoW/TF-IDF), embeddings dão a cada palavra um vetor curto e informativo.

### Exemplo:

- **Estáticos (Word2Vec, GloVe):** Cada palavra tem **um único vetor**, independentemente da frase -- “banco” sempre tem o mesmo vetor.
- **Contextuais (BERT e afins):** O vetor da palavra depende da **frase inteira** -- “banco” muda dependendo da frase (“banco de praça” vs “banco financeiro”).
- ✓ **Vantagens:** Capturam semântica e contexto; excelentes em NLP moderno.
- ✓ **Limitações:** Mais pesados; exigem mais cuidado/recursos.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*

- Transformam *pixels* brutos em representações numéricas que capturam padrões visuais relevantes, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.

**HOG (Histogram of Oriented Gradients):** Divide a imagem em pequenas regiões (células) e calcula histogramas da orientação das bordas (direções dos gradientes).

- **Captura contornos e formas (muito usado para detecção de pessoas/objetos em abordagens clássicas).**
- Bordas e orientação são estáveis e descrevem bem estrutura.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*

- Transformam *pixels* brutos em representações numéricas que capturam padrões visuais relevantes, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.

**SIFT (Scale-Invariant Feature Transform):** Detecta pontos-chave (keypoints) e cria descritores locais que são robustos a escala, rotação e alguma variação de iluminação.

- Captura “assinaturas” locais (cantos, interseções, texturas) sendo úteis para correspondência entre imagens, panorama, tracking, reconhecimento clássico.

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*

- Transformam *pixels* brutos em representações numéricas que capturam padrões visuais relevantes, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.

**CNN (Convolutional Neural Network):** Aprende automaticamente filtros/convoluções:

- **Camadas Iniciais:** bordas e texturas,
- **Camadas Intermediárias:** partes (olhos, rodas),
- **Camadas Profundas:** conceitos semânticos (faces, gatos, carros).

→ Portanto, a própria rede aprende quais atributos extrair para a tarefa.

- ✓ **Aplicação:** Transfer learning (usar uma CNN pré-treinada e extrair o vetor de uma camada como feature).



# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Séries temporais (áudio):** *Mel-frequency cepstral coefficients* (MFCCs), Transformada rápida de Fourier (FFT), Espectogramas, Transformada de *Wavelet*.

- Transformam os sinais em características que capturam propriedades espectrais ou de tempo-frequência.

**FFT (Transformada Rápida de Fourier):** Converte o sinal do domínio do tempo para o domínio da frequência.

- **Captura** quais frequências estão presentes e com que intensidade.
- ✓ **Limitação:** Não diz bem quando cada frequência ocorre (perde informação temporal fina).

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Séries temporais (áudio):** *Mel-frequency cepstral coefficients* (MFCCs), Transformada rápida de Fourier (FFT), Espectrogramas, Transformada de *Wavelet*.

- Transformam os sinais em características que capturam propriedades espectrais ou de tempo-frequência.

**Espectrograma (tempo–frequência):** Aplica FFT em janelas curtas ao longo do tempo (STFT), gerando uma “imagem”:

- Eixo x: tempo
- Eixo y: frequência
- Cor/intensidade: energia

→ Mostra como o conteúdo espectral muda no tempo (fala, música, ruídos).

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Séries temporais (áudio):** *Mel-frequency cepstral coefficients* (MFCCs), Transformada rápida de Fourier (FFT), Espectogramas, Transformada de *Wavelet*.

- Transformam os sinais em características que capturam propriedades espectrais ou de tempo-frequência.

**MFCC (Mel-Frequency Cepstral Coefficients):** Em fala/áudio, o que costuma diferenciar sons (vogais, fonemas, instrumentos) não é só “quais frequências existem”, mas **como a energia se distribui ao longo das frequências** (o *envelope spectral*).

- Os MFCCs são uma forma compacta de representar esse envelope.
- Basicamente pode-se dizer que o MFCC pega o espectro (FFT em janelas), aplica uma escala perceptual (**Mel**) que imita o ouvido humano e comprime e decorrelaciona gerando poucos coeficientes (os MFCCs).

**Captura características do “timbre” e forma espectral da fala sendo clássico em reconhecimento de fala.**

# PRÉ-PROCESSAMENTO DE DADOS

## → Técnicas de Extração de Atributos

**Séries temporais (áudio):** *Mel-frequency cepstral coefficients* (MFCCs), Transformada rápida de Fourier (FFT), Espectrogramas, Transformada de *Wavelet*.

- Transformam os sinais em características que capturam propriedades espectrais ou de tempo-frequência.

**Wavelet:** A Wavelet faz uma análise tempo–frequência como o espectrograma, mas com uma diferença crucial:

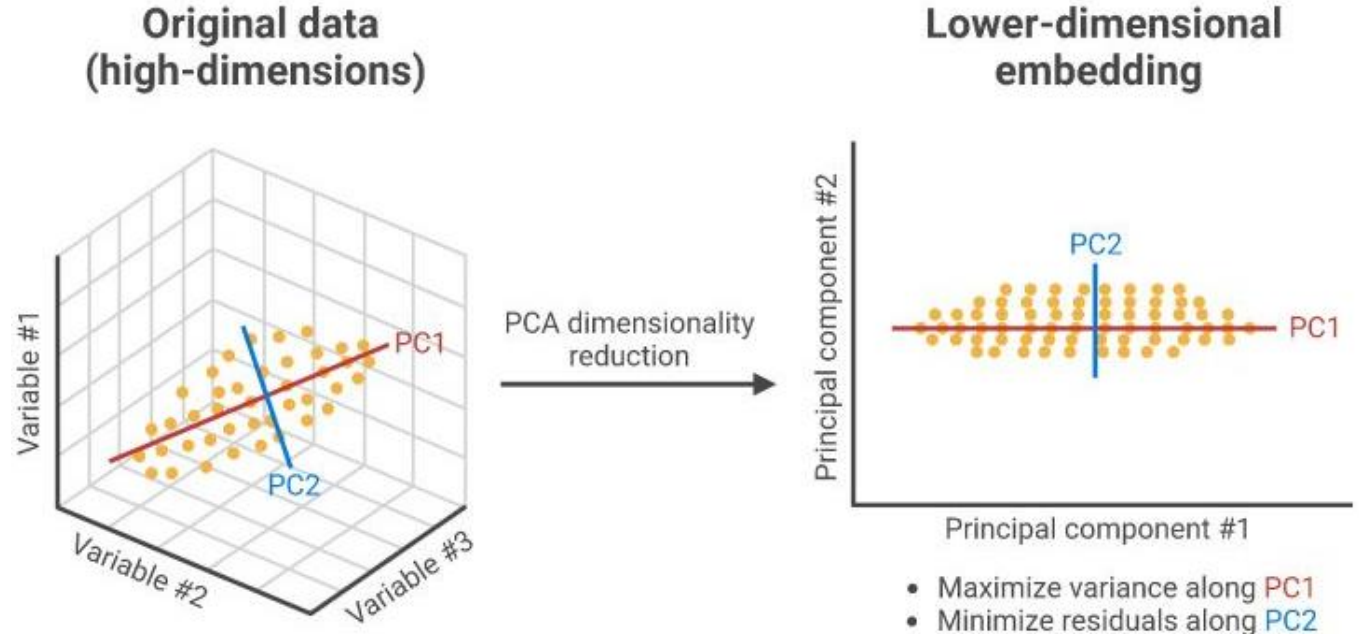
- Para **altas frequências** (eventos rápidos), ela usa **janelas curtas** → ótima **resolução temporal** (localiza “quando” aconteceu).
- Para **baixas frequências** (variações lentas), ela usa **janelas longas** → ótima **resolução em frequência** (distingue melhor frequências próximas).

**Excelente para sinais com eventos rápidos e transientes (batidas, falhas, picos) e para dados não estacionários.**

# PRÉ-PROCESSAMENTO DE DADOS

## → ANÁLISE DE COMPONENTES PRINCIPAIS (PCA)

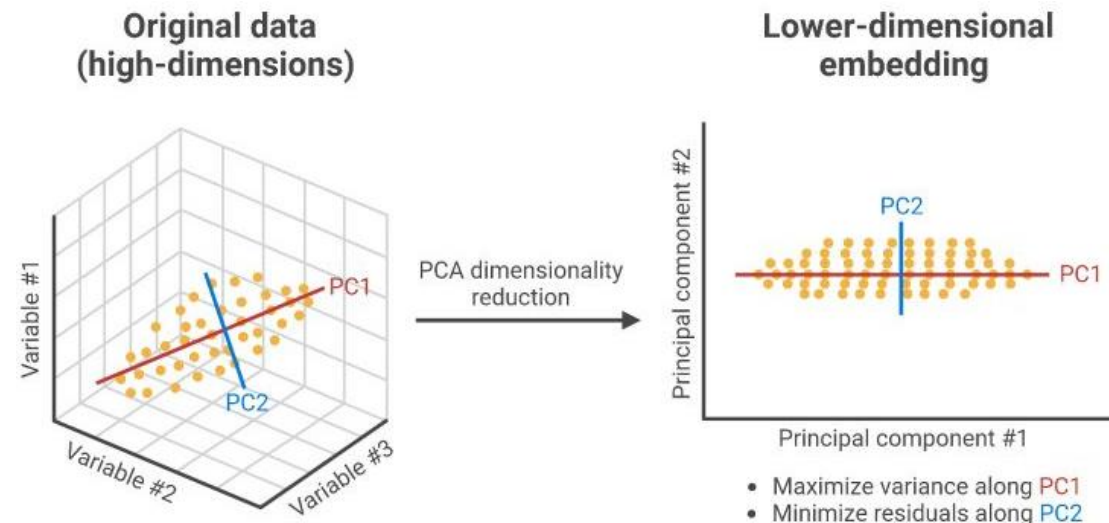
- O PCA projeta dados de alta dimensão em um espaço de menor dimensão.
- PCA funciona encontrando novas direções (componentes principais) que capturam o máximo de variância dos dados e **projetando** os dados nessas direções para reduzir a dimensionalidade com mínima perda de informação



# PRÉ-PROCESSAMENTO DE DADOS

## → ANÁLISE DE COMPONENTES PRINCIPAIS (PCA)

- O algoritmo encontra as direções que melhor descrevem como os dados estão espalhados (i.e., máxima variância).
- Na figura, PC1 é a direção onde os dados variam mais.
- Já PC2 é a direção ortogonal à PC1 com a segunda maior variância.
- Mantendo PC1, preserva-se a maior parte da informação.
- PC2 representa variação residual (menor importância).





# PRÉ-PROCESSAMENTO DE DADOS

## → ANÁLISE DE COMPONENTES PRINCIPAIS (PCA)

As PCs são ordenadas por variância.

- A matriz de **projeção** é ordenada da componente de maior variância para a menor.

Cada componente é uma **combinação linear** dos atributos originais.

O PCA **assume relações lineares entre os atributos** (traça linhas retas para capturar as variâncias).

- Se os dados originais seguem um padrão curvo (como uma senoide, meia lua ou um círculo), o PCA vai ter um desempenho bem ruim.
- Usar técnicas não lineares como Kernel PCA e t-SNE.

Deve ser aplicado após a separação do dados em conjuntos de treinamento, validação e teste para evitar vazamento.

# PRÉ-PROCESSAMENTO DE DADOS

## → ANÁLISE DE COMPONENTES PRINCIPAIS (PCA)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Escalonar os dados antes do PCA (é obrigatório que os atributos tenham a mesma variância)
X_scaled = StandardScaler().fit_transform(X)
```

```
# Extrair 2 componentes
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)
```

**OBS.:** “n\_components” pode ser também um número entre 0 e 1 especificando o mínimo de variância explicada desejada.

# PRÉ-PROCESSAMENTO DE DADOS

## → SELEÇÃO DE ATRIBUTOS

Processo de selecionar um subconjunto de atributos que contribuem para a melhoria do desempenho do modelo.

### Vantagens:

- ✓ **Reduz *overfitting*:** Modelos mais simples com menos atributos tendem a generalizar melhor para novos dados.
- ✓ **Reduz custo computacional:** Menos dados significam algoritmos que processam mais rápido.
- ✓ **Facilita interpretação:** É mais fácil explicar um modelo que utiliza 5 variáveis do que um que utiliza 500.

# PRÉ-PROCESSAMENTO DE DADOS

## → SELEÇÃO DE ATRIBUTOS

Processo de selecionar um subconjunto de atributos que contribuem para a melhoria do desempenho do modelo.

**Existem três tipos principais de técnicas de seleção de atributos:**

**Filtragem:** Baseados em medidas estatísticas dos dados, como correlação.

- **Exemplos:** Correlação de Pearson/Spearman, Teste qui-quadrado, limiar de variância, ganho de informação, informação mútua.

**Encapsulamento:** Usa o próprio modelo de ML para testar subconjuntos de variáveis e avaliar a performance.

- **Exemplo:** Eliminação recursiva de atributos.

**Embarcado:** A seleção ocorre durante o processo de treinamento do próprio algoritmo. É integrada (i.e., embarcada) no treinamento.

- **Exemplos:** Regressão LASSO e árvores de decisão.

# PRÉ-PROCESSAMENTO DE DADOS

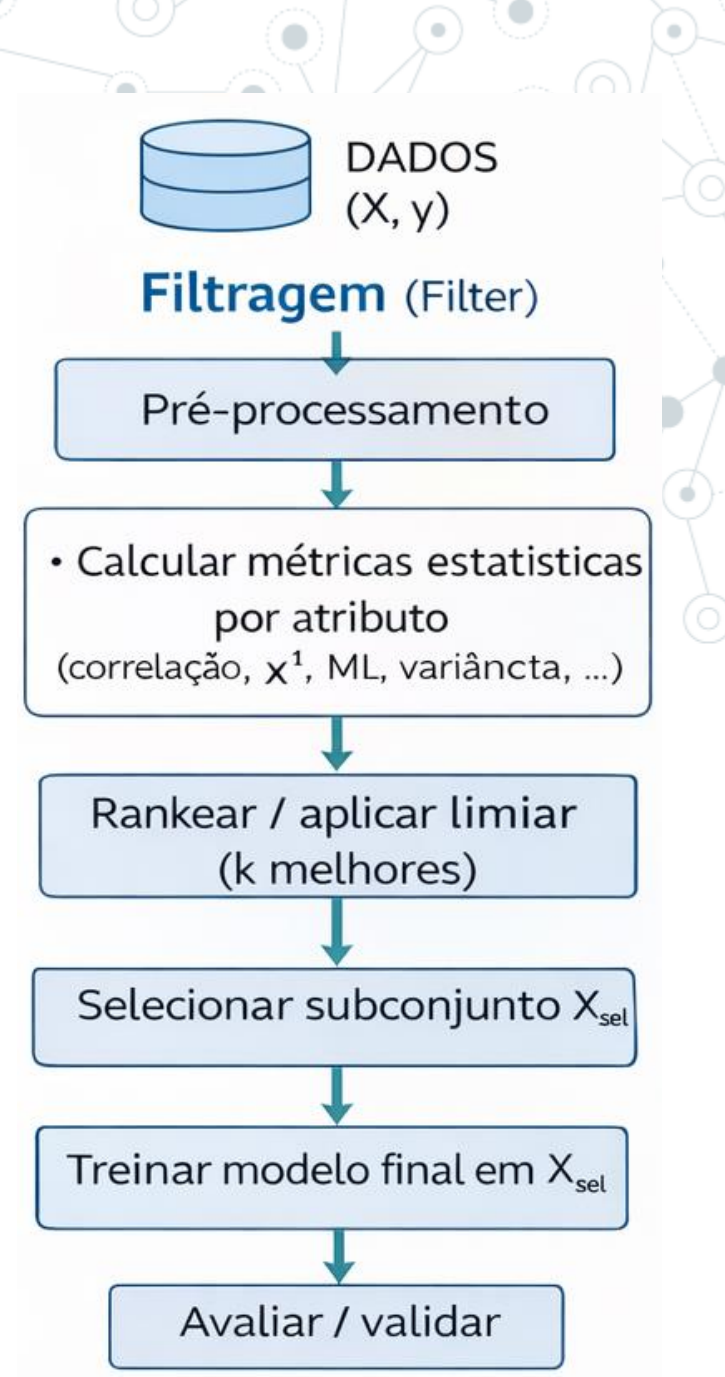
## → SELEÇÃO DE ATRIBUTOS

### Métodos de Filtro

- Seleciona atributos **antes** de treinar o modelo, usando **critérios estatísticos** (sem depender de um algoritmo específico).

**“Quais variáveis parecem relevantes olhando só para os dados?”**

- São rápidos, simples e independentes do modelo, mas podem ignorar interações relevantes entre atributos (devido à independência do modelo) e dependem da escolha adequada da métrica (e.g., relações lineares e não lineares).



# PRÉ-PROCESSAMENTO DE DADOS

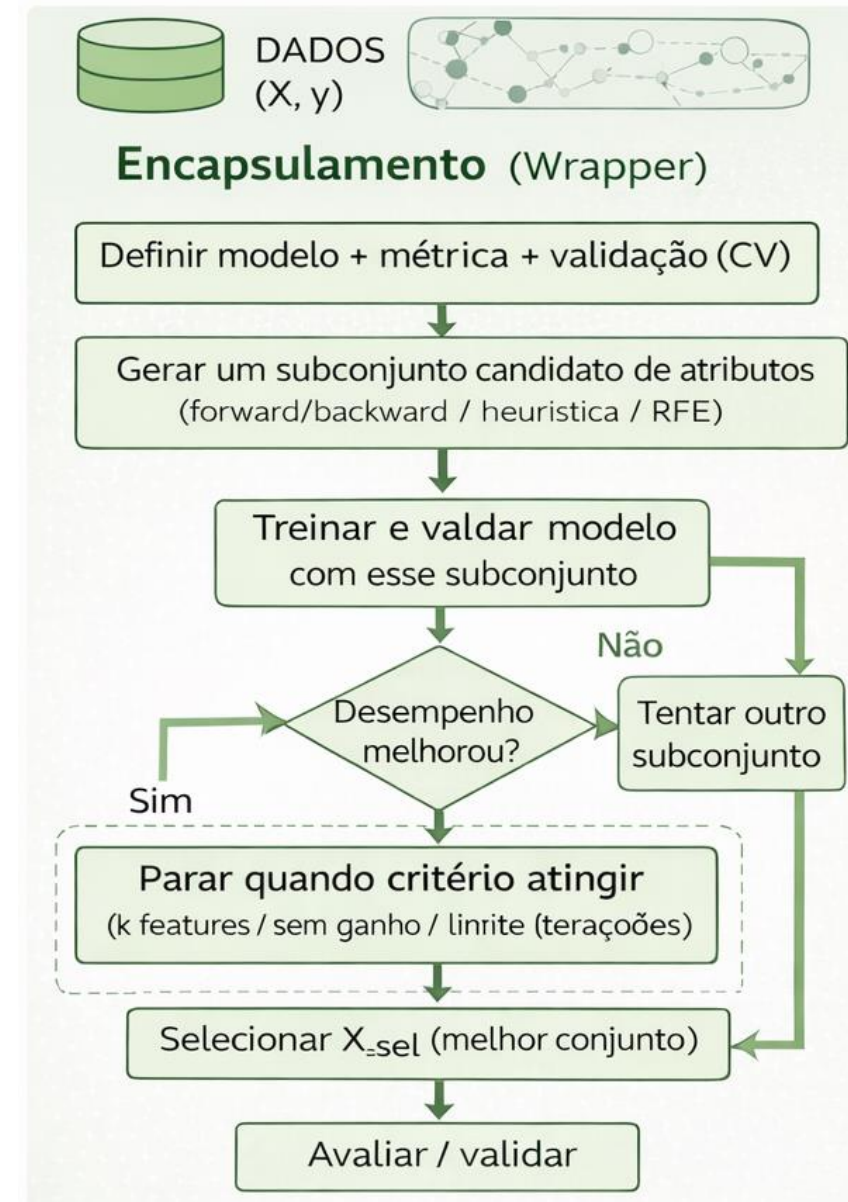
## → SELEÇÃO DE ATRIBUTOS

### Métodos de Encapsulamento

- Usa o **desempenho do próprio modelo** para avaliar subconjuntos de atributos. Ou seja, ele **treina e valida várias vezes** com combinações diferentes.

**“Quais atributos fazem o meu modelo performar melhor de verdade?”**

- Otimizam os atributos para um modelo específico e podem ter desempenho superior em comparação aos métodos de filtro, mas são computacionalmente caros e suscetíveis ao *overfitting*, pois podem ajustar excessivamente os atributos a um modelo específico.





# PRÉ-PROCESSAMENTO DE DADOS

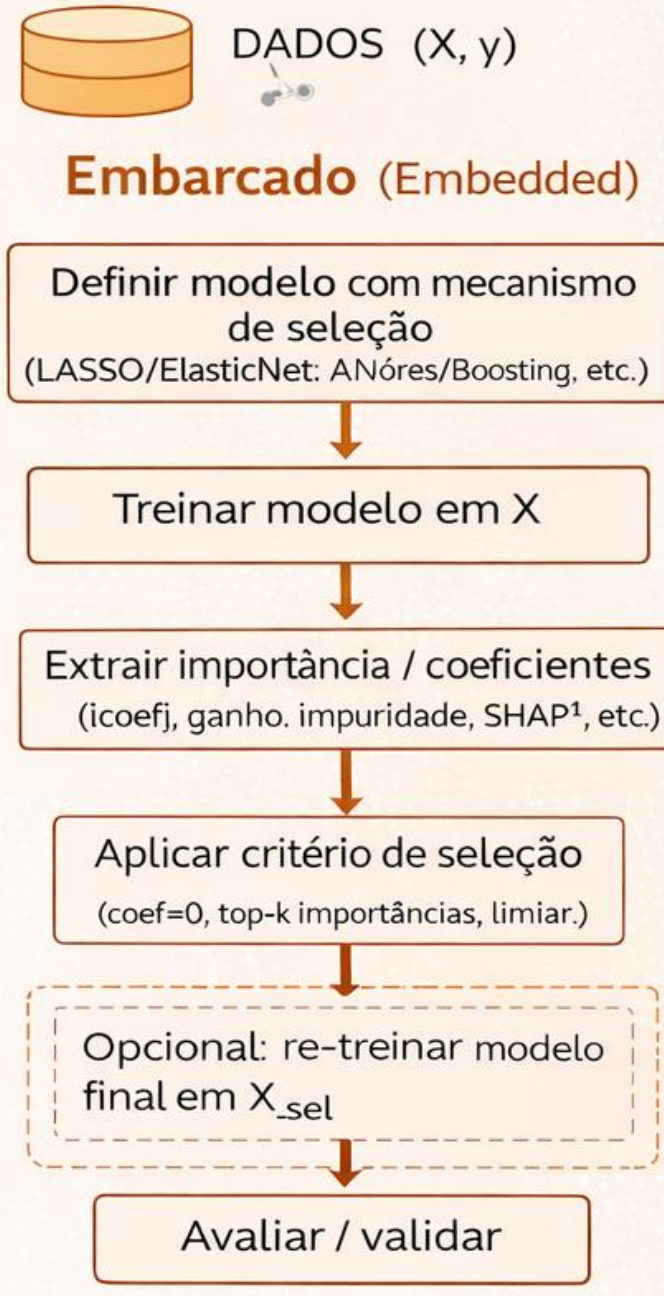
## → SELEÇÃO DE ATRIBUTOS

### Métodos Embarcados

- A seleção acontece **durante o treinamento**, porque o próprio algoritmo tem mecanismos que **penalizam** ou **priorizam** atributos.

**“O modelo já incorpora seleção de atributos na forma como aprende.”**

- Selecionam atributos durante o treinamento do modelo de forma eficiente, apresenta equilíbrio entre desempenho e tempo para seleção, mas têm interpretabilidade limitada e não são suportados por todos os algoritmos.



# PRÉ-PROCESSAMENTO DE DADOS

## → SELEÇÃO DE ATRIBUTOS

O módulo `sklearn.feature_selection` implementa vários métodos de seleção de atributos, dentre eles estão:

- Limiar de variância (`VarianceThreshold`)
- Eliminação recursiva de atributos (RFE)
- Selecione os K melhores atributos de acordo com um *score* (`SelectKBest`)
- Seleção usando modelos com penalidade L1 ou baseados em árvore (`SelectFromModel`)
- Seleção de atributos sequencial (`SequentialFeatureSelector`)

# PRÉ-PROCESSAMENTO DE DADOS

## → SELEÇÃO DE ATRIBUTOS

```
from sklearn.feature_selection import VarianceThreshold
```

```
# Remove features com variância <= 0.1
```

```
selector = VarianceThreshold(threshold=0.1)
```

```
X_reduced = selector.fit_transform(df)
```

```
# Quais features foram mantidas?
```

```
# (Descobre as colunas que foram mantidas)
```

```
selected_features = df.columns[selector.get_support()]
```

```
print("\nFeatures selecionadas:")
```

```
# Imprime a lista (Index) com os nomes das colunas que ficaram após o filtro de variância.
```

```
print(selected_features)
```

- Cria o “seletor”.
- `threshold=0.1` define que serão removidas as colunas cuja variância seja menor que 0.1.
- A variância mede o quanto um atributo “varia”. Se ele quase não muda, tende a trazer pouca informação para o modelo.

- `fit(df)`: calcula a variância de cada coluna do `df` e decide quais passam no limiar.
- `transform(df)`: retorna uma nova matriz apenas com as colunas mantidas.

`selector.get_support()` retorna uma máscara booleana do tipo:

- `True` = coluna mantida
- `False` = coluna removida

# PRÉ-PROCESSAMENTO DE DADOS

## → EXTRAÇÃO x SELEÇÃO DE ATRIBUTOS

**Extração de atributos:** Você cria NOVOS atributos a partir dos dados existentes.

→ **Ex.:** data\_hora → dia\_da\_semana, hora, é\_fim\_de\_semana, mês.

**Seleção de atributos:** Você ESCOLHE quais atributos manter dentre os disponíveis.

→ **Ex.:** dentre 200 colunas, manter as 30 mais relevantes.

# PRÉ-PROCESSAMENTO DE DADOS

## → EXEMPLO:

- [Exemplo: pre\\_processing.ipynb](#)
- **Github:** Prof. Felipe Figueiredo



**Obrigada pela Atenção!**

**Dúvidas?**

[victoria.souto@Inatel.br](mailto:victoria.souto@Inatel.br)